Delegates and Events

NET 022 - Advanced C# Programming

Prepared By: Supun Kandaudahewa
supun.kandaudahewa@humber.ca

HUMBER

WE ARE HUMBER

Agenda

- Understanding Delegates
  - Single-Cast Delegates
  - Multi-Cast Delegates
- Events
- Anonymous Methods
- Lambda Expressions and Inline Lambda Expressions
- Predefined Generic Delegates
  - Func
  - Action
  - Predicate
- Event Handlers
- Coding Exercises

Delegate

- “Delegate type” is a “type” that represents (references to) methods with a **particular parameter list** and a **return type**.

- The delegate object stores reference/s (address/es) of a specific method/s of a specific class with compatible parameters and return type.

- All delegate types are derived from “System.Delegate” class.

- To use delegates, you need to:
  - Create a Delegate type.
  - Create a Delegate Object.
  - Invoke method using Delegate Object.

[3] https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/

Delegate Contd.

- Creating Delegate Type.

Creating Delegate Type
public delegate <ReturnType> DelegateTypeName(param1, param2, ….);

- Creating Delegate Object.

Creating Delegate Object
DelegateTypeName referenceVariable=new DelegateTypeName(MethodName);

- Invoke method using Delegate Object.

Invoking the Delegate Object
referenceVariable.Invoke(arg1, arg2, ….);

Types of Delegates

- Single-Cast Delegates
  - Contains reference of only one method.
  - When called, it directly invokes the referenced method.

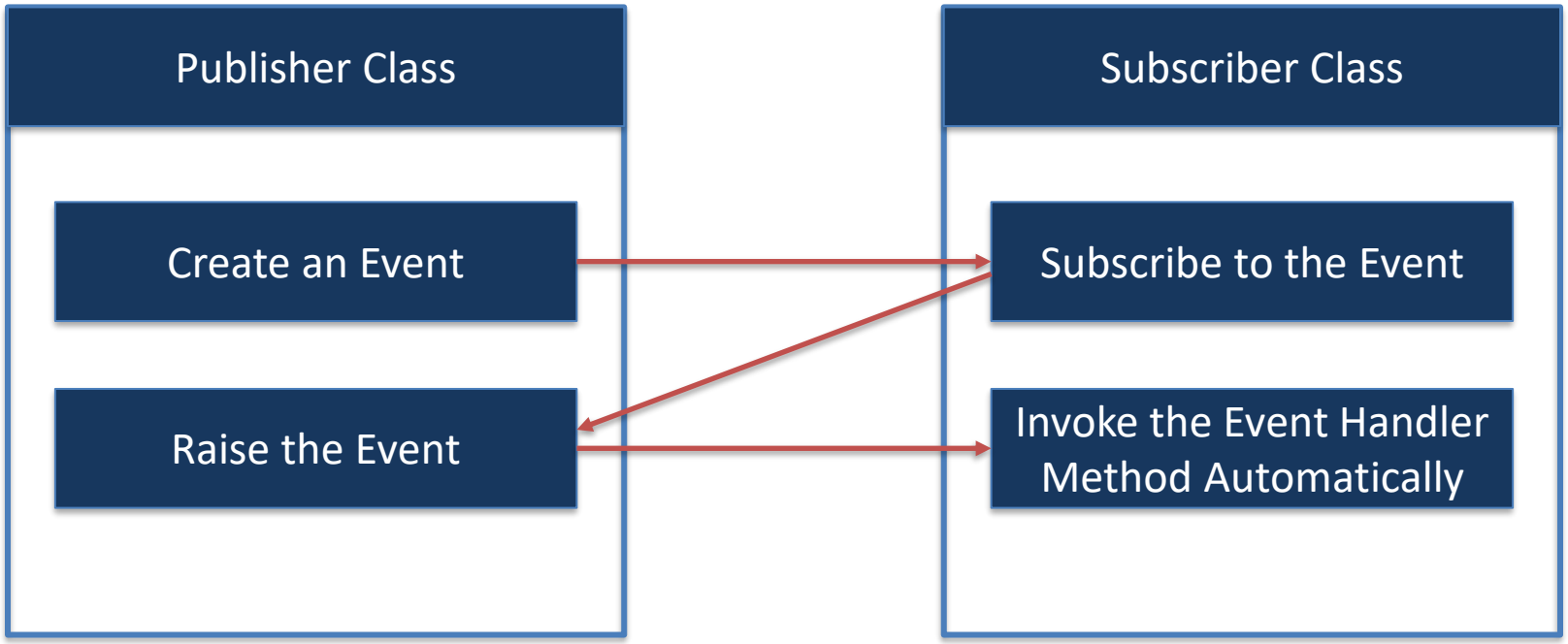
- Multi-Cast Delegates
  - Contains references to multiple methods.
  - When called, it invokes all the referenced method, one by one in a sequence.
  - All methods **parameters and return types should be the same**. (Functionality can be different).
  - Delegate functions are added using the "+=" operator and removed using "-="

Events

- Event is a multi-cast delegate that stores references of **one or more methods**. Invokes them every time when the event is raised.
- Events enable a class or object to notify other classes or objects when something of interest occurs.
- The event can only be raised in the same class in which the event was created.
- Event follows a publisher-subscriber model. Let's talk more about the publisher-subscriber model in the next slide.

 https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/events/

Events Contd.



Creating an Event

```
public event MyDelegateType MyEvent;
```

Creating an Event

- Create a Delegate.

- Publisher Class Activities
  - Create an Event in the Publisher Class.
  - Raise the event in the Publisher Class.

- Subscriber Class Activities
  - Create Event Handler Method.

- Subscribe to the event

Case Study –Loosely Coupled Systems Using Delegates and Events

OrderProcessor Class

```
Public Class OrderProceser
{
    public void ProcessOrder(List<Items> items, Customer customer)
    {
        //The order processing logic goes here and now order is ready!

        //After processing the order should send an SMS to customer
        SmsService.SendMessage(orderId, customer);

        //Another service needs to be added → an Email Service
        EmailService.SendMessage(orderId, customer);
    }
}
```

Case Study

SMS Service Class (Email Service also mimics the same behavior)

```
Public Static Class SmsService
{
    public static void SendMessage(string orderId, Customer customer)
    {
    // Mimicking the message sending behavior
    Console.WriteLine($”Dear {customer.CustomerName}, your order
no: {orderId} is now ready for pick up! ”);
    }
}
```

Case Study

Item Class

```
Public Class Item
{
        public Guid ItemId { get; set; }
        public string ItemName { get; set; }
        public string ItemDescription { get; set; }
        public decimal Price { get; set; }
}
```

Customer Class

```
Public Class Customer
{
        public Guid CustomerId { get; set; }
        public string CustomerName { get; set; }
        public string Email { get; set; }
        public string MobileNumber { get; set; }
        public string Address { get; set; }
}
```

Anonymous Methods

- As the name suggests, anonymous methods are methods without a name/“name-less methods”.
- Anonymous methods can be invoked using the delegate variable or an event.
- So, we need not create a “named method” in class level. Saves time and more concise syntax.
- Anonymous methods are mainly used for event handlers.
- These methods can't contain jump statements like goto, break, continue etc.

Subscribe to an Event with Anonymous Methods

```
EventName + = delegate (param1, param 2, …)
{
      //method body goes here
}
```

Lambda Expressions

- In anonymous methods writing the delegate keyword when subscribing to an event does not provide any value.
- We use the => operator when writing Lambda expressions. It is also called as “goes to” or “goes into” operator.
- When the logic can be written in one line (small calculations or condition checks) we can use **inline lambda expressions**.

Lambda Expressions

```
EventName + = (param1, param 2, …) =>
{
      //method body goes here
}
```

Inline Lambda Expressions

```
EventName + = (param1, param 2, …) => condition or calculation
```

Type Inference of Lambdas

- You often do not have to specify a type for the input parameters because the compiler can infer the type based on the lambda body.
- The general rules for lambdas are as follows:
  - The lambda must contain the same number of parameters as the delegate type.
  - Each input parameter in the lambda must be **implicitly convertible** to its corresponding delegate parameter.
  - The return value of the lambda (if any) must be **implicitly convertible** to the delegate's return type.

Variable Scope of Lambdas

- Lambdas can refer to outer variables: **Concept of "Capture"**
- Variables introduced **within a lambda expression are not visible in the outer method**.
- A return statement in a lambda expression does not cause the enclosing method to return.
- A lambda expression **cannot contain** a `goto` statement, `break` statement, or `continue` statement that is inside the lambda function, if the jump statement's target is outside the block

Anonymous Methods and Lambda Expressions Code Samples

- Anonymous Methods - https://dotnetfiddle.net/MeCsiQ

- Lambda Expressions - https://dotnetfiddle.net/0SFhTq

Func<T, TResult>

- Func is a **pre-defined generic-delegate**, which can be used to create events quickly.
- Func supports **0 to 16 input parameters**, and a **value needs to be returned**.

Creating a Delegate based on Func

```
public Func<param1DataType, param2DataType, returnType>  referenceVariable;
```

0 to 16 input parameters

Last one is the return type and is a must

Action<T>

- Action is also a **pre-defined generic-delegate**, which can be used to create events quickly.
- Action supports **0 to 16 input parameters**, and **it does not return any value (returns void).**

Creating a Delegate based on Action

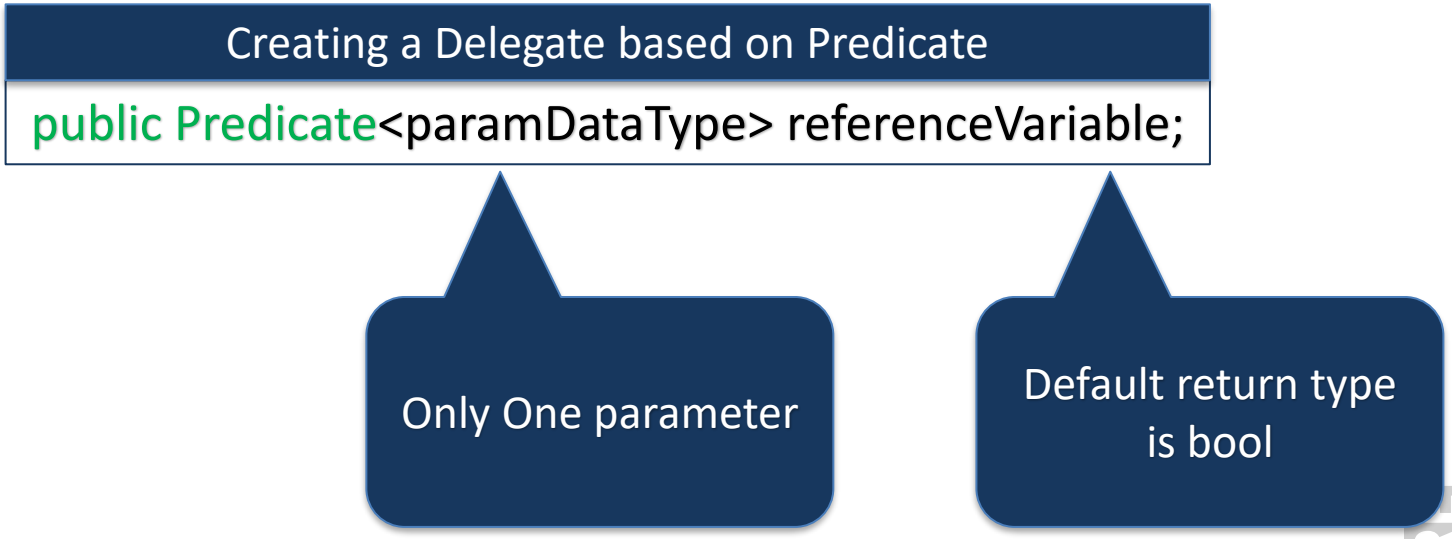
```
public Action<param1DataType, param2DataType, ….>  referenceVariable;
```

0 to 16 input parameters

[No return value] – Returns void

Predicate

- Predicate is also a **pre-defined generic-delegate**, which can be used to create events quickly. (Heavily used in LINQ)
- Predicate **must only have one (1) parameter**, and **it should always return a bool.**



Func, Action and Predicate Code Samples

- Func- https://dotnetfiddle.net/8IiDok
- Action - https://dotnetfiddle.net/iZwzr0
- Predicate - https://dotnetfiddle.net/2aI3sX

Event Handlers

- **EventHandler** is a pre-defined delegate type, which has two parameters called "**object sender**" and "**EventArgs”**; and no return (void return).

- **object sender** → Represents the source object, where the event is originally raised.

- **EventArgs e**: Represents additional parameters to pass to 'event handler method’.

Event Handlers Contd.

- .NET Framework includes the following built-in delegate types for the most common events. It is recommended you follow this pattern, but it's not mandatory.

  - **EventHandler**
    https://docs.microsoft.com/en-us/dotnet/api/system.eventhandler
  - **EventHandler<TEventArgs>**
    https://docs.microsoft.com/en-us/dotnet/api/system.eventhandler-1

- Events pass EventArgs or a derived class (event data).

Creating and EventArgs Class

- The **EventArgs** class is used in the signature of many delegates and event handlers.
- When custom data needs to be passed the **EventArgs** class can be extended.
- This class is deriving from the `**System.EventArgs**` Class.
- To use a custom **EventArgs** class, the delegate must reference the class in its signature.
- .NET includes a generic **EventHandler<T>** class that can be used instead of a custom delegate.

https://learn.microsoft.com/en-us/dotnet/api/system.eventargs?view=net-6.0

Building an Employee Onboarding Program

- Let's see about the typical activities relating to employee onboarding and let’s build the code to get things done.

    - Issue ID Card
    - Send Onboarding Email / Notify Employee
    - Update the DB with employee details
    - Prepare onboarding package
    - Arrange orientation training …….

Note: The code for this activity will be available on blackboard after the class.

THANK YOU.

HUMBER

WE ARE HUMBER